Proyecto: Visor de imágenes

Juan Camilo Guevara Osorio

Computación Gráfica
Francisco Alejandro Medina Aguirre

Universidad Tecnológica de Pereira
2026

Descripción del Proyecto:

Este proyecto tiene como objetivo desarrollar un visor de imágenes interactivo en Python que permita al usuario aplicar múltiples transformaciones sobre imágenes digitales. Se integran funcionalidades de carga, visualización, edición (brillo, contraste, rotación, filtros), fusión de imágenes, generación de histogramas y exportación de resultados. El proyecto promueve la aplicación integrada de todos los conceptos vistos durante el curso.

Objetivos:

- Desarrollar una interfaz gráfica amigable para la visualización y edición de imágenes.
- Aplicar técnicas de transformación y filtrado de imágenes con NumPy.
- Implementar herramientas de análisis gráfico como histogramas y canales RGB/CMY.
- Gestionar múltiples imágenes, operaciones de fusión y herramientas interactivas (zoom, rotación).
- Documentar adecuadamente la aplicación desarrollada.

Metodología:

El estudiante desarrollará de manera progresiva los componentes del visor, integrando gradualmente las funcionalidades básicas y avanzadas. El proyecto se desarrolla durante las últimas semanas del curso, con revisiones parciales y asesorías periódicas del docente.

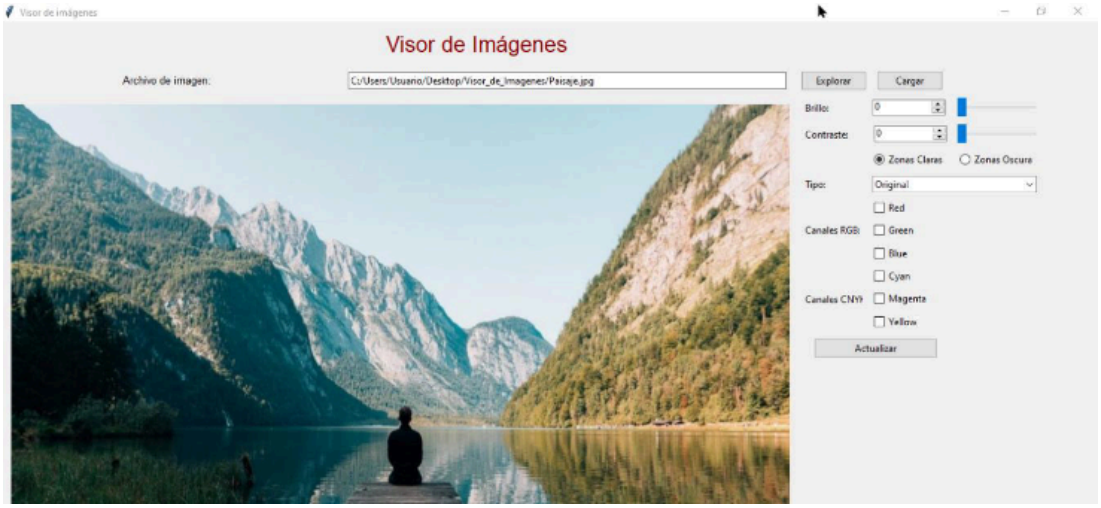
Recursos Necesarios:

- Computador con Python, Tkinter o PyQt5, NumPy, Matplotlib instalados
- Imágenes de prueba en formatos JPG, PNG y BMP
- Editor de código (VS Code, PyCharm, Jupyter)

Evaluación:

La evaluación se basa en la completitud, funcionalidad y calidad técnica del visor, así como en su interfaz, documentación y presentación oral.

Requisitos mínimos del visor



Resumen de requerimientos

- Tecnologías :
  - Python + tkinter para la interfaz.
  - numpy para todas las transformaciones de imagen.
  - matplotlib para histogramas.
  - Librería de lectura/escritura de imágenes.
- Interfaz principal :
  - Ventana “Visor de Imágenes”.
  - Campo de texto con la ruta de la imagen.
  - Botones:
    - Explorar (archivo).
    - Cargar (mostrar imagen).
    - Actualizar (aplicar todos los ajustes elegidos).
    - Guardar (guardar resultado).
    - Fusionar imágenes (cargar segunda imagen para fusión).
  - Sliders:
    - Brillo.
    - Contraste.
    - Rotación (0° a 360°).
    - Transparencia de fusión (0 a 1; visualmente 0–100%).
  - Filtros / opciones:
    - Zonas claras / zonas oscuras.
    - Checkboxes de canales RGB .
    - Checkboxes de canales CMY .
  - Funciones adicionales:
    - Zoom por coordenadas (X, Y) + factor de escala.
    - Binarización con umbral ajustable.
    - Histograma (ventana extra con matplotlib).
    - Negativo de imagen.
    - Fusión de dos imágenes con transparencia.
- Criterios de diseño :
  - Separar interfaz y procesamiento en clases diferentes.
  - Usar Frame de tkinter para organizar la ventana.
  - Trabajar con imágenes como arrays NumPy .

EL PROYECTO ES ENTREGADO JUNTO CON SU MANUAL DE USUARIO DONDE SE MUESTRA EL FUNCIONAMIENTO DEL PROGRAMA

Estructura del programa

INTERFAZ

Parte Superior

Esto se construye desde el módulo de ui mediante el método **`_build_layout`** asignando la ubicación, texto y métodos correspondientes a sus funciones:

```
## ---------- / Parte superior / ---------- ##

#titulo de programa
title_label = ttk.Label(
    title_frame,
    text="Visor de Imágenes",
    anchor="center",
    foreground="black",
    font=("Arial", 18, "bold"),
)
#centra horizontalmente el titulo
title_label.pack(fill=tk.X)

#frame que agrupa ruta y botones de archivo.
path_frame = ttk.Frame(self.root)
path_frame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=5)

path_label = ttk.Label(path_frame, text="Archivo de imagen:") #etiqueta para el campo de ruta
path_label.pack(side=tk.LEFT)

path_entry = ttk.Entry(path_frame, textvariable=self.image_path_var) #campo de texto para la ruta
path_entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=5)

#se deja expandible
path_entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=5)

explore_button = ttk.Button(path_frame, text="Explorar", command=self._on_explore) #boton para explorar el archivo
explore_button.pack(side=tk.LEFT, padx=2)

load_button = ttk.Button(path_frame, text="Cargar", command=self._on_load) #boton para cargar la imagen principal
load_button.pack(side=tk.LEFT, padx=2)

save_button = ttk.Button(path_frame, text="Guardar", command=self._on_save) #boton para guardar la imagen procesada
save_button.pack(side=tk.LEFT, padx=2)

#boton para fusionar imagenes
fusion_button = ttk.Button(
    path_frame,
    text="Fusionar Imágenes",
    command=self._on_load_second_image,
)
fusion_button.pack(side=tk.LEFT, padx=2)
```

Las funciones llamadas son las siguientes:

Botón de *Explorar*

```python
def _on_explore(self):
    """Abre el explorador de archivos para elegir la imagen principal."""
    filetypes = [("Imágenes", "*.jpg *.jpeg *.png *.bmp"), ("Todos los archivos", "*.*")] #extensiones permitidas
    filename = filedialog.askopenfilename(title="Seleccionar imagen", filetypes=filetypes) #abre el explorador de archivos
    if filename:
        self.image_path_var.set(filename) #actualiza la ruta visible
```

Botón de *Cargar*

```python
def _on_load(self):
    """Carga la imagen principal seleccionada y la muestra en pantalla."""
    path = self.image_path_var.get().strip() #obtiene la ruta visible
    if not path: #Si no hay ruta visible, muestra error
        messagebox.showerror("Error", "Seleccione un archivo de imagen.")
        return

    try:
        array = self.processor.load_main_image(path) #carga la imagen principal
    except Exception as exc:
        #si falla la carga, muestra error y cancela la operacion
        messagebox.showerror("Error", f"No se pudo cargar la imagen.\n{exc}")
        return

    self._show_array_on_canvas(array) #dibuja la imagen en el canvas
    self._on_update(show_errors=False) #actualiza los controles
```

Botón de *Guardar*

```python
def _on_save(self):
    """Guarda en disco la imagen procesada actual."""
    if self.processor.image_array is None: #si no hay imagen procesada, muestra error
        messagebox.showerror("Error", "No hay imagen para guardar.")
        return

    filetypes = [("Imagen PNG", "*.png"), ("Imagen JPG", "*.jpg *.jpeg"), ("Bitmap", "*.bmp")] #formatos permitidos

    #abre el cuadro de dialogo para elegir nombre y ruta de salida.
    filename = filedialog.asksaveasfilename(
        title="Guardar imagen",
        defaultextension=".png",
        filetypes=filetypes,
    )
    #si el usuario cancela, se abandona la operacion.
    if not filename:
        return

    try:
        self.processor.save_image(filename) #guarda la imagen en disco
    except Exception as exc:
        #en caso de algun error, muestra error y cancela la operacion
        messagebox.showerror("Error", f"No se pudo guardar la imagen.\n{exc}")
```

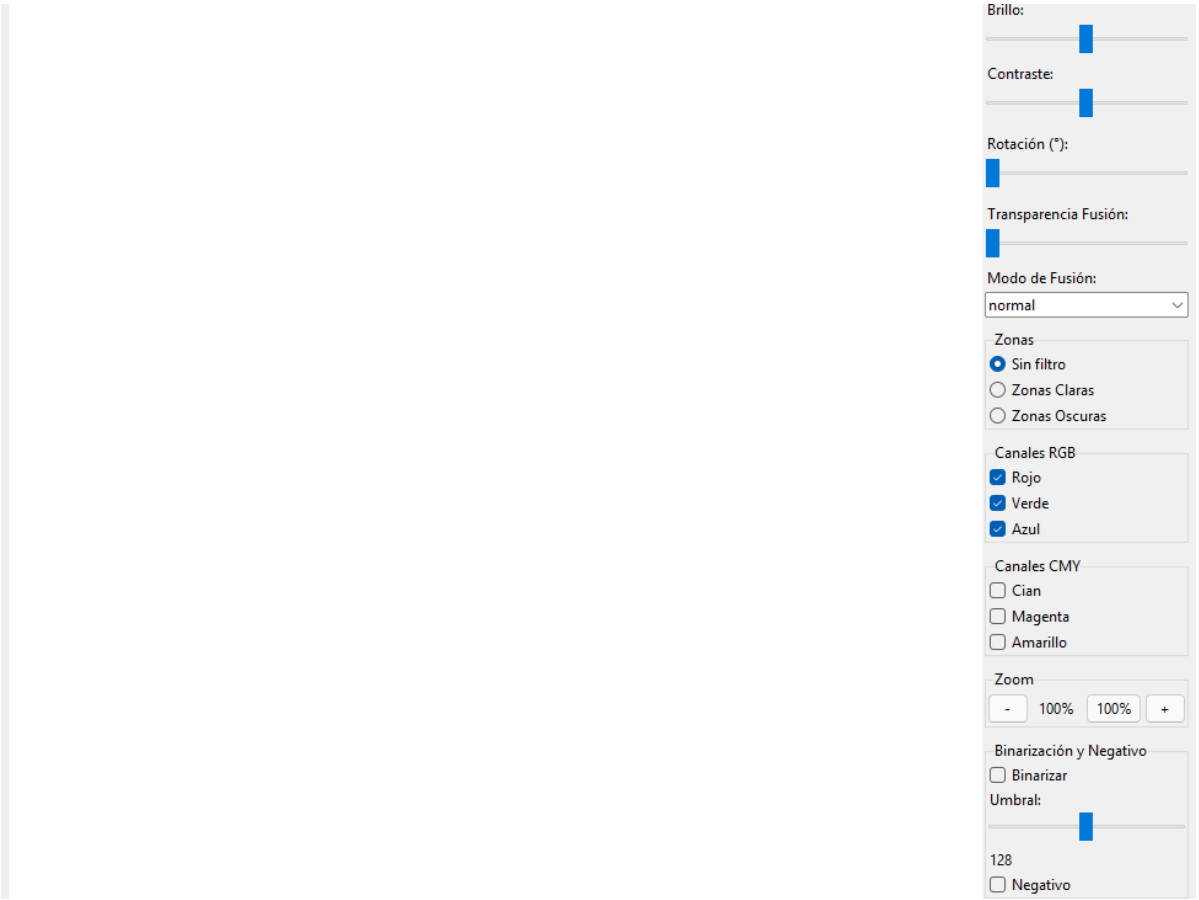
Botón de *Fusionar Imágenes*

```python
def _on_load_second_image(self):
    """Carga una segunda imagen para realizar fusion."""
    filetypes = [("Imágenes", "*.jpg *.jpeg *.png *.bmp"), ("Todos los archivos", "*.*")] #extensiones permitidas
    #abre el dialogo de seleccion para la imagen secundaria.
    filename = filedialog.askopenfilename(
        title="Seleccionar segunda imagen",
        filetypes=filetypes,
    )
    #si el usuario cancela, se abandona la operacion.
    if not filename:
        return

    try:
        self.processor.load_second_image(filename) #carga la imagen secundaria
    except Exception as exc:
        #si falla la carga, muestra error y cancela la operacion
        messagebox.showerror("Error", f"No se pudo cargar la segunda imagen.\n{exc}")
        return

    self._on_update(show_errors=False) #actualiza los controles
    messagebox.showinfo("Información", "Segunda imagen cargada para la fusión.")
```

Zona Principal



Se compone del canvas donde se muestra la imágen y el panel de edición, el panel tiene relación directa con el procesador de imágenes, el layout de crea de la siguiente manera:

Canvas para la imágen principal

```
main_frame = ttk.Frame(self.root)
main_frame.pack(side=tk.TOP, fill=tk.BOTH, expand=True, padx=10, pady=5)

image_frame = ttk.Frame(main_frame) #crea el frame para la imagen principal
image_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True) #ubica el frame a la izquierda y le permite crecer
image_frame.columnconfigure(0, weight=1) #le permite expandirse en la columna
image_frame.rowconfigure(0, weight=1) #le permite expandirse en la fila principal

self.canvas = tk.Canvas(image_frame, bg="white") #crea el canvas para la visualización
self.canvas.grid(row=0, column=0, sticky="nsew") #coloca el canvas llenando toda la celda disponible
self.canvas.bind("<Configure>", self._on_canvas_resize) #vincula el cambio de tamaño del canvas para redibujar la imagen
```

```python
### NAVEGACIÓN DE IMÁGEN ###
#barra horizontal para navegar la imagen ampliada
self.horizontal_scrollbar = ttk.Scrollbar(
    image_frame,
    orient=tk.HORIZONTAL,
    command=self.canvas.xview,
)
#barra vertical para navegar la imagen ampliada
self.vertical_scrollbar = ttk.Scrollbar(
    image_frame,
    orient=tk.VERTICAL,
    command=self.canvas.yview,
)
#conecta el canvas con ambas barras de desplazamiento
self.canvas.configure(
    xscrollcommand=self.horizontal_scrollbar.set,
    yscrollcommand=self.vertical_scrollbar.set,
)

self.horizontal_scrollbar.grid(row=1, column=0, sticky="ew") #coloca la barra horizontal en la fila inferior
self.vertical_scrollbar.grid(row=0, column=1, sticky="ns") #coloca la barra vertical en la columna derecha
self.horizontal_scrollbar.grid_remove() #oculta la barra horizontal mientras no sea necesaria
self.vertical_scrollbar.grid_remove() #oculta la barra vertical mientras no sea necesaria
```

Panel lateral

```python
#crea y ubica el frame para los controles lateral
controls_frame = ttk.Frame(main_frame, width=260)
controls_frame.pack(side=tk.RIGHT, fill=tk.Y)
```

Slider de brillo

```python
### BRILLO ###
#crea y ubica la etiqueta del control de brillo
brightness_label = ttk.Label(controls_frame, text="Brillo:")
brightness_label.pack(anchor="w", pady=(0, 2))

#slider para controlar el brillo de la imagen
brightness_scale = ttk.Scale(
    controls_frame,
    from_=-100,
    to=100,
    orient=tk.HORIZONTAL,
    variable=self.brightness_var,
    command=self._on_live_change,
)
brightness_scale.pack(fill=tk.X, pady=(0, 5))
```

Slider de contraste

```python
### CONTRASTE ###
#crea y ubica la etiqueta del control de contraste
contrast_label = ttk.Label(controls_frame, text="Contraste:")
contrast_label.pack(anchor="w", pady=(0, 2))
#slider para controlar el contraste de la imagen
contrast_scale = ttk.Scale(
    controls_frame,
    from_=-100,
    to=100,
    orient=tk.HORIZONTAL,
    variable=self.contrast_var,
    command=self._on_live_change,
)
contrast_scale.pack(fill=tk.X, pady=(0, 5))
```

Slider de rotación

```python
### ROTACIÓN ###
#crea y ubica la etiqueta del control de rotación
rotation_label = ttk.Label(controls_frame, text="Rotación (°):")
rotation_label.pack(anchor="w", pady=(5, 2))
#slider para controlar el angulo de rotación
rotation_scale = ttk.Scale(
    controls_frame,
    from_=0,
    to=360,
    orient=tk.HORIZONTAL,
    variable=self.rotation_var,
    command=self._on_live_change,
)
rotation_scale.pack(fill=tk.X, pady=(0, 5))
```

Slider de transparencia de fusión

```python
### TRANSPARENCIA FUSIÓN ###
#crea y ubica la etiqueta del control de fusion
fusion_label = ttk.Label(controls_frame, text="Transparencia Fusión:")
fusion_label.pack(anchor="w", pady=(5, 2))
#slider para controla cuanto participa la segunda imagen.
fusion_scale = ttk.Scale(
    controls_frame,
    from_=0.0,
    to=1.0,
    orient=tk.HORIZONTAL,
    variable=self.fusion_alpha_var,
    command=self._on_live_change,
)
fusion_scale.pack(fill=tk.X, pady=(0, 5))
```

Seleccionador de modo de fusión

```python
### MODO FUSIÓN ###
#crea y ubica la etiqueta del selector de modo de fusion
fusion_mode_label = ttk.Label(controls_frame, text="Modo de Fusión:")
fusion_mode_label.pack(anchor="w", pady=(0, 2))
#crea un combobox con los modos disponibles de superposicion
fusion_mode_combo = ttk.Combobox(
    controls_frame,
    textvariable=self.fusion_mode_var,
    values=["normal", "multiplicacion", "adicion", "diferencia"],
    state="readonly",
)
fusion_mode_combo.pack(fill=tk.X, pady=(0, 5))
#actualiza la imagen cuando el usuario cambia el modo
fusion_mode_combo.bind("<<ComboboxSelected>>", self._on_entry_commit)
```

Seleccionador de zonas de iluminación

```python
### ZONAS ###
#crea y ubica el frame para los controles de zonas
zones_frame = ttk.LabelFrame(controls_frame, text="Zonas")
zones_frame.pack(fill=tk.X, pady=5)

#crea la opcion para no aplicar filtro de zonas.
no_zone_radio = ttk.Radiobutton(
    zones_frame,
    text="Sin filtro",
    value="none",
    variable=self.zone_mode_var,
    command=self._on_live_change,
)
no_zone_radio.pack(anchor="w") #ubica la opcion dentro del grupo
#crea la opcion para resaltar zonas claras.
light_radio = ttk.Radiobutton(
    zones_frame,
    text="Zonas Claras",
    value="light",
    variable=self.zone_mode_var,
    command=self._on_live_change,
)
light_radio.pack(anchor="w") #ubica la opcion dentro del grupo
#crea la opcion para resaltar zonas oscuras.
dark_radio = ttk.Radiobutton(
    zones_frame,
    text="Zonas Oscuras",
    value="dark",
    variable=self.zone_mode_var,
    command=self._on_live_change,
)
dark_radio.pack(anchor="w") #ubica la opcion dentro del grupo
```

Controlador de canales RGB

```python
### RGB ###
#crea y ubica el frame para los controles de canales RGB
rgb_frame = ttk.LabelFrame(controls_frame, text="Canales RGB")
rgb_frame.pack(fill=tk.X, pady=5)

#crea el check del canal rojo.
rgb_r = ttk.Checkbutton(
    rgb_frame,
    text="Rojo",
    variable=self.rgb_red_var,
    command=self._on_live_change,
)
rgb_r.pack(anchor="w") #ubica el check dentro del grupo
#crea el check del canal verde.
rgb_g = ttk.Checkbutton(
    rgb_frame,
    text="Verde",
    variable=self.rgb_green_var,
    command=self._on_live_change,
)
rgb_g.pack(anchor="w") #ubica el check dentro del grupo
#crea el check del canal azul.
rgb_b = ttk.Checkbutton(
    rgb_frame,
    text="Azul",
    variable=self.rgb_blue_var,
    command=self._on_live_change,
)
rgb_b.pack(anchor="w") #ubica el check dentro del grupo
```

Controlador de canales CMY

```python
### CMY ###
#crea y ubica el frame para los controles de canales CMY
cmy_frame = ttk.LabelFrame(controls_frame, text="Canales CMY")
cmy_frame.pack(fill=tk.X, pady=5)

#crea el check del canal cian.
cmy_c = ttk.Checkbutton(
    cmy_frame,
    text="Cian",
    variable=self.cmy_cyan_var,
    command=self._on_live_change,
)
cmy_c.pack(anchor="w") #ubica el check dentro del grupo
#crea el check del canal magenta.
cmy_m = ttk.Checkbutton(
    cmy_frame,
    text="Magenta",
    variable=self.cmy_magenta_var,
    command=self._on_live_change,
)
cmy_m.pack(anchor="w") #ubica el check dentro del grupo
#crea el check del canal amarillo.
cmy_y = ttk.Checkbutton(
    cmy_frame,
    text="Amarillo",
    variable=self.cmy_yellow_var,
    command=self._on_live_change,
)
cmy_y.pack(anchor="w") #ubica el check dentro del grupo
```

Control del zoom

```python
### ZOOM ###
#crea y ubica el frame para los controles de zoom
zoom_frame = ttk.LabelFrame(controls_frame, text="Zoom")
zoom_frame.pack(fill=tk.X, pady=5)

#crea y ubica un frame interno para alinear botones y etiqueta.
zoom_controls_frame = ttk.Frame(zoom_frame)
zoom_controls_frame.pack(fill=tk.X, pady=2)

#crea y ubica el boton para reducir el zoom.
zoom_out_button = ttk.Button(zoom_controls_frame, text="-", width=4, command=self._zoom_out)
zoom_out_button.pack(side=tk.LEFT)

#crea y ubica la etiqueta que muestra el porcentaje actual de zoom.
self.zoom_label = ttk.Label(zoom_controls_frame, text="100%", anchor="center")
self.zoom_label.pack(side=tk.LEFT, expand=True, padx=6)

#crea y ubica el boton que restaura el zoom al 100 por ciento.
zoom_reset_button = ttk.Button(zoom_controls_frame, text="100%", width=6, command=self._zoom_reset)
zoom_reset_button.pack(side=tk.LEFT, padx=2)

#crea y ubica el boton para aumentar el zoom.
zoom_in_button = ttk.Button(zoom_controls_frame, text="+", width=4, command=self._zoom_in)
zoom_in_button.pack(side=tk.LEFT)
```

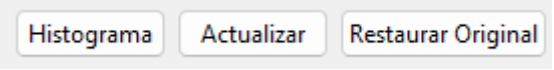
Control de binarización

```python
### BINARYIZACION Y NEGATIVO ###
#crea y ubica el frame para los controles de binarizacion y negativo
binary_frame = ttk.LabelFrame(controls_frame, text="Binarización y Negativo")
binary_frame.pack(fill=tk.X, pady=5)
#crea y ubica el check para activar la binarizacion.
binary_check = ttk.Checkbutton(
    binary_frame,
    text="Binarizar",
    variable=self.apply_binary_var,
    command=self._on_live_change,
)
binary_check.pack(anchor="w") #muestra el check en el grupo.
#crea y ubica la etiqueta que describe el slider de umbral.
threshold_label = ttk.Label(binary_frame, text="Umbral:")
threshold_label.pack(anchor="w")
#crea y ubica el slider que define el umbral de binarizacion.
threshold_scale = ttk.Scale(
    binary_frame,
    from_=0,
    to=255,
    orient=tk.HORIZONTAL,
    variable=self.threshold_var,
    command=self._on_live_change,
)
threshold_scale.pack(fill=tk.X, pady=(0, 4)) #lo muestra en el grupo.
```

Control del blanco y negro

```
#crea y ubica la etiqueta que refleja numericamente el umbral.
self.threshold_value_label = ttk.Label(binary_frame, text="128")
self.threshold_value_label.pack(anchor="w")
#crea el check para activar el negativo.
negative_check = ttk.Checkbutton(
    binary_frame,
    text="Negativo",
    variable=self.apply_negative_var,
    command=self._on_live_change,
)
negative_check.pack(anchor="w") #muestra el check en el grupo.
```

Zona inferior



Son opciones adicionales para la imágen:

- Genera un histograma de la imágen actual teniendo en cuenta los filtros y modificaciones realizadas
- Actualiza forzosamente la imágen en caso de algún atasco tanto en el procesamiento de la imágen como en la generación del histograma
- Restaura el estado original de la imágen

Código

```
## ---------- / Parte Inferior / ---------- ##

#crea y ubica el frame inferior para acciones globales.
bottom_frame = ttk.Frame(self.root)
bottom_frame.pack(side=tk.BOTTOM, fill=tk.X, padx=10, pady=5)

#crea y ubica el boton que abre la ventana del histograma.
histogram_button = ttk.Button(bottom_frame, text="Histograma", command=self._on_histogram)
histogram_button.pack(side=tk.LEFT, padx=5)

#crea y ubica el boton que fuerza una actualizacion manual.
update_button = ttk.Button(bottom_frame, text="Actualizar", command=self._on_update)
update_button.pack(side=tk.LEFT)

#crea y ubica el boton para volver a la imagen original.
restore_button = ttk.Button(
    bottom_frame,
    text="Restaurar Original",
    command=self._on_restore_original,
)
restore_button.pack(side=tk.LEFT, padx=5)
```

Funciones utilizadas a lo largo del código

_on_update(): Recalcula la imágen en base a los controles actuales

```
def _on_update(self, show_errors=True):
    """Lee los controles actuales y recalcula la imagen mostrada."""
    #si aun no hay imagen principal, no se intenta procesar nada.
    if self.processor.original_image_array is None:
        return

    try:
        Delegate to agent
        #crea un diccionario simple con todos los parametros.
        config = self._build_processing_config()
        #aplica la cadena completa de transformaciones.
        array = self.processor.apply_transformations(config)
    except Exception as exc:
        #si se permite mostrar errores, se informa el problema al usuario.
        if show_errors:
            messagebox.showerror("Error", f"No se pudo actualizar la imagen.\n{exc}")
        return

    #actualiza la imagen visible en el canvas.
    self._show_array_on_canvas(array)
```

_on_histogram(): Llama al procesador de imágenes para cargar el histograma de la imágen actual

```
def _on_histogram(self):
    """Solicita al procesador mostrar el histograma actual."""
    try:
        #genera y muestra el histograma de la imagen procesada.
        self.processor.show_histogram()
    except Exception as exc:
        #si algo falla, se muestra un cuadro de error.
        messagebox.showerror("Error", f"No se pudo generar el histograma.\n{exc}")
```

_on_restore_original(): Restaura la imágen a su estado original en base a una copia limpia creada por el procesador de imágenes, a su vez que los controles.

```
def _on_restore_original(self):
    """Restaura la imagen original y reinicia los controles."""
    try:
        #recupera una copia limpia de la imagen original.
        array = self.processor.restore_original_image()
    except Exception as exc:
        #si falla la restauracion, se informa el problema.
        messagebox.showerror("Error", f"No se pudo restaurar la imagen.\n{exc}")
        return

    #reinicia los controles al estado base del programa.
    self._reset_controls()
    #dibuja la imagen original en pantalla.
    self._show_array_on_canvas(array)
```

_build_processing_config(): Crea un diccionario con todos los valores de la configuración para que el procesador de imágenes pueda trabajarlos de mejor forma.

```python
def _build_processing_config(self):
    """Empaqueta todos los valores de la interfaz en un diccionario."""
    return {
        "brightness": int(round(self.brightness_var.get())), #redondea y convierte el brillo a entero.
        "contrast": int(round(self.contrast_var.get())), #redondea y convierte el contraste a entero.
        "rotation": int(round(self.rotation_var.get())), #redondea la rotacion para trabajar con grados enteros.
        "fusion_alpha": float(self.fusion_alpha_var.get()), #obtiene el valor actual del control de fusion.
        "fusion_mode": self.fusion_mode_var.get(), #obtiene el modo de la superposicion elegido.
        "zone_mode": self.zone_mode_var.get(), #obtiene el filtro actual de zonas.
        "rgb_red": self.rgb_red_var.get(), #consulta si el canal rojo esta activo.
        "rgb_green": self.rgb_green_var.get(), #consulta si el canal verde esta activo.
        "rgb_blue": self.rgb_blue_var.get(), #consulta si el canal azul esta activo.
        "cmy_cyan": self.cmy_cyan_var.get(), #consulta si el canal cian esta activo.
        "cmy_magenta": self.cmy_magenta_var.get(), #consulta si el canal magenta esta activo.
        "cmy_yellow": self.cmy_yellow_var.get(), #consulta si el canal amarillo esta activo.
        "zoom_scale": float(self.zoom_scale_var.get()), #obtiene el factor de zoom de la vita.
        "threshold": int(self.threshold_var.get()), #obtiene el umbral de binarizacion.
        "apply_binary": self.apply_binary_var.get(), #indica si se aplica binarizacion.
        "apply_negative": self.apply_negative_var.get(), #indica si se aplica el negativo.
    }
```

_on_live_change(): Actualiza en tiempo real las acciones de los sliders

```python
def _on_live_change(self, _value=None):
    """Actualiza la vista cuando cambia un slider o check."""
    self._refresh_status_labels() #refresca las etiquetas numericas visibles.
    self._on_update(show_errors=False) #actualiza los controles
```

_on_entry_commit(): Actualiza en tiempo real las acciones de las listas

```python
def _on_entry_commit(self, _event=None):
    """Actualiza la vista al confirmar un valor de lista."""
    self._on_live_change() #actualiza la vista en vivo
```

_on_canvas_resize(): Redibuja la imágen cuando cambia el tamaño del canvas

```python
def _on_canvas_resize(self, _event=None):
    """Redibuja la imagen cuando cambia el tamano del canvas."""
    if self.processor.image_array is not None: #si hay una imagen procesada cargada
        self._show_array_on_canvas(self.processor.image_array) #dibuja la imagen en pantalla.
```

Controles de zoom: Tanto el aumento, decremento y reseto

```python
def _zoom_in(self):
    """Aumenta el zoom de la vista en pasos de 25 por ciento."""
    current_scale = self.zoom_scale_var.get() #obtiene el factor de zoom actual.
    self.zoom_scale_var.set(min(4.0, current_scale + 0.25)) #incrementa el zoom en 25%
    self._on_live_change()

def _zoom_out(self):
    """Disminuye el zoom de la vista en pasos de 25 por ciento."""
    current_scale = self.zoom_scale_var.get() #obtiene el factor de zoom actual.
    self.zoom_scale_var.set(max(0.25, current_scale - 0.25))#decrementa el zoom en 25%
    self._on_live_change()

def _zoom_reset(self):
    """Restablece el zoom al 100 por ciento."""
    self.zoom_scale_var.set(1.0) #fija el zoom al 100%
    self._on_live_change()
```

_reset_controls(): Restaura todos los controles

```python
def _reset_controls(self):
    """Devuelve todos los controles al estado inicial del visor."""
    self.brightness_var.set(0) #restaura el brillo al valor neutro.
    self.contrast_var.set(0) #restaura el contraste al valor neutro.
    self.rotation_var.set(0) #reinicia la rotacion a cero grados.
    self.fusion_alpha_var.set(0.0) #elimina la presencia de la segunda imagen en la fusion.
    self.fusion_mode_var.set("normal") #vuelve a usar el modo de fusion basico.
    self.zone_mode_var.set("none") #desactiva cualquier resaltado de zonas.
    self.rgb_red_var.set(True) #reactiva el canal rojo.
    self.rgb_green_var.set(True) #reactiva el canal verde.
    self.rgb_blue_var.set(True) #reactiva el canal azul.
    self.cmy_cyan_var.set(False) #desactiva el canal cian.
    self.cmy_magenta_var.set(False) #desactiva el canal magenta.
    self.cmy_yellow_var.set(False) #desactiva el canal amarillo.
    self.zoom_scale_var.set(1.0) #restaura el zoom al 100%.
    self.threshold_var.set(128) #restaura el umbral al valor medio.
    self.apply_binary_var.set(False) #desactiva la binarizacion.
    self.apply_negative_var.set(False) #desactiva el negativo.
    self._refresh_status_labels() #actualiza las etiquetas asociadas a estos valores.
```

_show_array_on_canvas(): Convierte el arreglo de la imágen en una impágen visible y la dibuja en el canvas considerando su tamaño y necesidad de navegación

```python
def _show_array_on_canvas(self, array):
    """Convierte un arreglo en imagen visible y la dibuja en el canvas."""
    if array is None: #si no hay una imagen valida
        return

    height = array.shape[0] #obtiene la altura original de la imagen.
    width = array.shape[1] #obtiene el ancho original de la imagen.
    canvas_width = self.canvas.winfo_width() #consulta el ancho actual disponible en el canvas.
    canvas_height = self.canvas.winfo_height() #consulta el alto actual disponible en el canvas.

    #si el canvas es muy pequeño, se usa un valor predeterminado para el dibujo.
    if canvas_width <= 1 or canvas_height <= 1:
        canvas_width = 800
        canvas_height = 500

    base_scale = min(canvas_width / width, canvas_height / height) #calcula la escala base que hace encajar la imagen en el canvas.
    zoom_scale = float(self.zoom_scale_var.get()) #obtiene el factor de zoom actual.
    display_scale = base_scale * zoom_scale #calcula la escala final combinando ajuste base y zoom.
    #calcula el ancho y alto final que tendra la imagen en pantalla.
    new_width = max(1, int(width * display_scale))
    new_height = max(1, int(height * display_scale))

    image = Image.fromarray(array) #convierte el arrego de NumPy en una imagen PIL.
    image = image.resize((new_width, new_height), Image.LANCZOS) #redimensiona la imagen segun la escala calculada.
    self.display_image = ImageTk.PhotoImage(image) #convierte la imagen PIL en un objeto compatible con Tkinter.

    #limpia el canvas
    self.canvas.delete("all")
    #decide si hacen falta scrollbars cuando el zoom supera el tamano visible.
    use_scrollbars = zoom_scale > 1.0 and (new_width > canvas_width or new_height > canvas_height)

    if use_scrollbars: #si la imagen ampliada ya no cabe completamente en el canvas
        #muestra las barras de desplazamiento
        self._set_scrollbars_visibility(True)
        self.canvas.create_image(0, 0, image=self.display_image, anchor="nw") #dibuja la imagen desde la esquina superior izquierda
        self.canvas.configure(scrollregion=(0, 0, new_width, new_height)) #actualiza la region desplazable del canvas
        self._center_canvas_view(new_width, new_height, canvas_width, canvas_height) #centra la vista inicial
    else: #si no se necesitan scrollbars
        self._set_scrollbars_visibility(False) #oculta las barras de desplazamiento
        #dibuja la imagen centrada dentro del canvas
        self.canvas.create_image(
            canvas_width // 2,
            canvas_height // 2,
            image=self.display_image,
            anchor="center",
        )
        self.canvas.configure(scrollregion=(0, 0, canvas_width, canvas_height)) #redefine la region visible para el tamano actual del canvas
        self.canvas.xview_moveto(0) #vuelve el desplazamiento horizontal a 0
        self.canvas.yview_moveto(0) #vuelve el desplazamiento vertical a 0
```

_set_scrollbars_visibility(): Define si se ven o no las barras de desplazamiento en la imágen

```python
def _set_scrollbars_visibility(self, visible):
    """Muestra u oculta las barras de desplazamiento del visor."""
    if visible: #muestra las barras de desplazamiento
        self.horizontal_scrollbar.grid()
        self.vertical_scrollbar.grid()
    else: #oculta las barras de desplazamiento
        self.horizontal_scrollbar.grid_remove()
        self.vertical_scrollbar.grid_remove()
```

_center_canvas_view(): Centra la vista del canvas dentro de la imágen ampliada

```python
def _center_canvas_view(self, content_width, content_height, view_width, view_height):
    """Centra la ventana visible del canvas sobre la imagen ampliada."""
    if content_width > view_width: #si la imagen ampliada sobrepasa el ancho visible del canvas
        max_offset_x = content_width - view_width #calcula el desplazamiento maximo posible en horizontal para centrar la vista
        offset_x = max_offset_x / 2.0 #tomar la mitad del desplazamiento maximo para centrar horizontalmente
        self.canvas.xview_moveto(offset_x / content_width) #mueve la vista horizontalmente usando proporcion del contenido para centrar horizontalmente
    else: #si no sobrepasa el ancho visible del canvas
        self.canvas.xview_moveto(0) #vuelve el desplazamiento horizontal a 0

    if content_height > view_height: #si la imagen ampliada sobrepasa el alto visible del canvas
        max_offset_y = content_height - view_height #calcula el desplazamiento maximo posible en vertical para centrar la vista
        offset_y = max_offset_y / 2.0 #tomar la mitad del desplazamiento maximo para centrar verticalmente usando proporcion del contenido.
        self.canvas.yview_moveto(offset_y / content_height) #mueve la vista verticalmente usando proporcion del contenido para centrar verticalmente
    else: #si no sobrepasa el alto visible del canvas
        self.canvas.yview_moveto(0) #vuelve el desplazamiento vertical a 0
```

PROCESADOR DE IMÁGENES

El procesador de imágenes se encarga de aplicar los filtros y trabajar la imágen directamente, no tiene ningún efecto en la interfaz ni en el control de archivos. Es de donde la UI obtiene los métodos para la manipulación de la imágen.

Elementos iniciales

```python
def __init__(self):
    """Inicializa las referencias de trabajo del procesador."""
    self.original_image_array = None #imagen original sin modificaciones
    self.image_array = None #ultima version procesada de la imagen principal
    self.second_image_array = None #imagen para fusionar con la principal
```

Inicialmente, el procesador define los elementos con los que va a trabajar.

Carga de imágenes

```python
def load_main_image(self, path):
    """Carga la imagen principal desde disco."""
    array = mpimg.imread(path) #lee la imagen con matplotlib
    array = self._ensure_uint8_rgb(array) #normaliza la imagen a RGB uint8
    self.original_image_array = array #almacena una copia de la imagen original sin modificaciones
    self.image_array = array.copy() #almacena la imágena a trabajar
    return self.image_array

def load_second_image(self, path):
    """Carga una segunda imagen para la fusion."""
    array = mpimg.imread(path) #lee la imagen con matplotlib
    array = self._ensure_uint8_rgb(array) #normaliza la imagen a RGB uint8
    self.second_image_array = array #almacena la imagen
    return array
```

El procesador carga las imágenes desde la ruta dada y las convierte en un array normalizado para trabajar con ellas. Crea una copia de la imágen original para poderla restaurar en un futuro

Restauración de la imágen

```python
def restore_original_image(self):
    """Restaura la imagen principal a su estado original."""
    #si aun no hay imagen original, no se puede restaurar
    if self.original_image_array is None:
        raise ValueError("No hay imagen original para restaurar.")

    self.image_array = self.original_image_array.copy() #restaura la imágena a su estado original sin modificaciones
    return self.image_array
```

Reemplaza la imágen visible por la copia de la imágen original

Guardado de imágen

```python
def save_image(self, path, use_second=False):
    """Guarda la imagen principal o secundaria en disco."""
    if use_second: #usa la segunda imagen para guardarla si es solicitado
        array = self.second_image_array
    else: #si no, usa la imagen principal procesada
        array = self.image_array

    if array is None: #si hay una imagen disponible para guardar, lanza una excepcion
        raise ValueError("No hay imagen para guardar.")

    plt.imsave(path, array) #guarda la imágena en la ruta indicada con matplotlib
```

Guarda la imágen en una ruta indicada utilizando matplotlib. La opción para guardar la segunda imágen no es abierta al usuario, sino más bien es una opción de desarrollador.

Aplicación de las transformaciónes

```python
def apply_transformations(self, config):
    """Aplica todas las transformaciones segun la configuracion recibida."""
    if self.original_image_array is None: #si no hay una imagen original, no se puede transformar
        raise ValueError("Primero debe cargar una imagen principal.")

    image = self.original_image_array.copy() #trabaja con una copia de la imagen original sin modificaciones, para evitar acumulaciones

    #aplica los ajustes
    image = self._apply_brightness(image, config.get("brightness", 0)) #brilo
    image = self._apply_contrast(image, config.get("contrast", 0)) #contraste
    image = self._apply_zone_filter(image, config.get("zone_mode", "none")) #zonas de luz
    #canales RGB.
    image = self._apply_rgb_channels(
        image,
        config.get("rgb_red", True),
        config.get("rgb_green", True),
        config.get("rgb_blue", True),
    )
    #canales CMY
    image = self._apply_cmy_channels(
        image,
        config.get("cmy_cyan", False),
        config.get("cmy_magenta", False),
        config.get("cmy_yellow", False),
    )
    if config.get("apply_binary", False): #binarización segun umbral
        image = self._apply_binarization(image, config.get("threshold", 128))

    if config.get("apply_negative", False): #negativo
        image = self._apply_negative(image)

    image = self._apply_rotation(image, config.get("rotation", 0)) #rotación

    image = self._apply_fusion( #fusión con la segunda imagen
        image,
        config.get("fusion_alpha", 0.0),
        config.get("fusion_mode", "normal"),
    )

    self.image_array = image #guarda la imagen ya procesada
    return self.image_array
```

Aplica todas las transformaciones a la imagen en base la configuración obtenida por el diccionario otorgado por la UI.

Histograma

```python
def show_histogram(self):
    """Genera y muestra el histograma de la imagen actual."""
    if self.image_array is None: #si no hay una imagen procesada, no se puede generar el histograma
        raise ValueError("No hay imagen cargada para generar el histograma.")

    image = self.image_array
    luminance = self._calculate_luminance(image) #calcula la luminancia aproximada para el canal de intensidad

    #crea la ventana de Matplotlib para el histograma.
    plt.figure("Histograma de la imagen", figsize=(10, 6))
    plt.clf() #limpia la figura por si ya existia previamente
    plt.hist(image[:, :, 0].ravel(), bins=256, color="red", alpha=0.4, label="Rojo") #distribución del canal rojo
    plt.hist(image[:, :, 1].ravel(), bins=256, color="green", alpha=0.4, label="Verde") #distribución del canal verde
    plt.hist(image[:, :, 2].ravel(), bins=256, color="blue", alpha=0.4, label="Azul") #distribución del canal azul
    plt.hist(luminance.ravel(), bins=256, color="black", alpha=0.5, label="Luminosidad") #distribución de la luminancia

    #lanza el gráfico
    plt.title("Distribución de colores")
    plt.xlabel("Intensidad")
    plt.ylabel("Frecuencia")
    plt.legend()
    plt.tight_layout()
    plt.show()
```

El procesador de imágenes toma cada canal y lo convierte en un arreglo de 1 dimensión para poder mostrarlo correctamente en el histograma que se muestra como un gráfico de matplotlib incluyendo la luminancia.

Brillo

```python
def _apply_brightness(self, image, brightness):
    """Aumenta o disminuye el brillo de la imagen."""
    delta = (float(brightness) / 100.0) * 255.0 #variación de intensidad real
    adjusted = image.astype(np.float32) + delta #suma la variación a cada pixel usando float para evitar desbordes
    return np.clip(adjusted, 0, 255).astype(np.uint8) #recorta el resultado al rango valido y se vuelve a uint8
```

Aplica las modificaciones de brillo a la imágen, convirtiendo los valores del slider en un porcentaje real de intensidad de luz y reconvierte el formato del número para que el cálculo se realice de manera más precisa.

Contraste

```python
def _apply_contrast(self, image, contrast):
    """Modifica el contraste separando o acercando tonos al valor medio."""
    factor = max(0.0, (100.0 + float(contrast)) / 100.0) #factor multiplicativo
    adjusted = (image.astype(np.float32) - 128.0) * factor + 128.0 #contraste modificado
    return np.clip(adjusted, 0, 255).astype(np.uint8) #limita el resultado al rango valido y se vuelve a uint8
```

Modifica el contraste de la imágen calculando la distancia de cada píxel con el tono medio, igualmente alternando entre formatos para realizar cálculos más precisos.

Zonas de luminancia

```python
def _apply_zone_filter(self, image, zone_mode):
    """Resalta zonas claras u oscuras usando la luminancia."""
    if zone_mode == "none": #si el modo es none, la imagen se devuelve sin cambios
        return image

    #calcula la luminancia para clasificar las zonas como float
    luminance = self._calculate_luminance(image)
    result = image.astype(np.float32).copy()

    if zone_mode == "light": #si se quieren destacar zonas claras
        mask = luminance >= 170 #se crea una mascara alta
        result[mask] = np.clip(result[mask] * 1.15, 0, 255) #intensifica las zonas claras
        result[~mask] = result[~mask] * 0.35 #atenúa el resto de la imágen
    elif zone_mode == "dark": #si se quieren destacar zonas oscuras
        mask = luminance <= 85 #se crea una mascara baja
        result[mask] = np.clip(result[mask] * 1.35, 0, 255) #intensifica las zonas oscuras
        result[~mask] = result[~mask] * 0.45 #atenúa el resto de la imágen

    return np.clip(result, 0, 255).astype(np.uint8) #limita el resultado al rango valido y se vuelve a uint8
```

Calcula la luminosidad de cada pixel y los almacena en una matriz de booleanos en base a si superan el umbral definido y resalta aquellos pertenecientes a la zona seleccionada

Canales RGB

```python
def _apply_rgb_channels(self, image, red, green, blue):
    """Activa o desactiva directamente los canales RGB."""
    result = image.copy() #trabaja una copia de la imagen original
    if not red: #desactiva el canal rojo
        result[:, :, 0] = 0
    if not green: #desactiva el canal verde
        result[:, :, 1] = 0
    if not blue: #desactiva el canal azul
        result[:, :, 2] = 0
    return result
```

Activa o desactiva los canales RGB seleccionados

Canales CMY

```python
def _apply_cmy_channels(self, image, cyan, magenta, yellow):
    """Construye una visualizacion simple basada en combinaciones CMY."""
    if not any([cyan, magenta, yellow]): #si no hay ning canal CMY activo, no se altera la imagen
        return image

    result = np.zeros_like(image) #crea una imagen vacia del mismo tamaño que la original para almacenar el resultado
    if cyan: #si cian esta activo, se conservan verde y azul
        result[:, :, 1] = np.maximum(result[:, :, 1], image[:, :, 1])
        result[:, :, 2] = np.maximum(result[:, :, 2], image[:, :, 2])
    if magenta: #si magenta esta activo, se conservan rojo y azul
        result[:, :, 0] = np.maximum(result[:, :, 0], image[:, :, 0])
        result[:, :, 2] = np.maximum(result[:, :, 2], image[:, :, 2])
    if yellow: #si amarillo esta activo, se conservan rojo y verde
        result[:, :, 0] = np.maximum(result[:, :, 0], image[:, :, 0])
        result[:, :, 1] = np.maximum(result[:, :, 1], image[:, :, 1])
    return result
```

Activa o desactiva los canales CMY seleccionados, trabajando con combinaciones de canales RGB

Binarización

```python
def _apply_binarization(self, image, threshold):
    """Convierte la imagen a blanco y negro segun un umbral."""
    threshold = int(np.clip(threshold, 0, 255)) #fuerza el umbral al rango valido entre 0 y 255
    luminance = self._calculate_luminance(image) #calcula la luminancia para trabajar en una sola intensidad
    binary = np.where(luminance >= threshold, 255, 0).astype(np.uint8) #asigna blanco o negro segun si cada pixel supera el umbral
    return np.stack([binary, binary, binary], axis=-1) #replica el resultado en los tres canales para conservar formato RGB
```

Aplica el filtro de binarización según un umbral asignado que varía entre 0 y 255 y asigna blanco o negro según si la luminancia de cada pixel supera este umbral

Negativo

```python
def _apply_negative(self, image):
    """Invierte los colores de la imagen."""
    return 255 - image #complemento de cada pixel
```

Aplica filtro negativo a la imágen reemplazando cada pixel por su complemento

Rotación

```python
def _apply_rotation(self, image, rotation):
    """Rota la imagen usando PIL para simplificar la implementacion."""
    angle = float(rotation) % 360.0 #normaliza el ángulo al rango de 0 a 360 grados
    if angle == 0: #si el ángulo es 0, la imagen se devuelve sin cambios
        return image

    pil_image = Image.fromarray(image) #transforma la imagen NumPy a una imagen PIL
    rotated = pil_image.rotate(-angle, expand=True, fillcolor=(255, 255, 255)) #rota la imagen sin recortarla
    return np.array(rotated, dtype=np.uint8) #retorna la imágen rotada en formato de NumPy
```

Utiliza la librería Pillow para rotar la imágen sin recortarla, reemplazando el color del fondo vacío por color blanco

Fusión de imágenes

```python
def _apply_fusion(self, image, fusion_alpha, fusion_mode):
    """Combina la imagen principal con una segunda imagen usando varios modos."""
    if self.second_image_array is None: #si no hay segunda imagen, no se aplica fusión
        return image

    alpha = float(np.clip(fusion_alpha, 0.0, 1.0)) #fuerza el alpha al rango valido entre 0 y 1
    second = self._resize_to_shape(self.second_image_array, image.shape[1], image.shape[0]) #redimensiona la segunda imagen para que coincida con la principal
    base = image.astype(np.float32) #convierte la imagen principal a float para operar con seguridad
    overlay = second.astype(np.float32) #convierte la segunda imagen a float para operar con seguridad

    #modos de fusión:
    if fusion_mode == "multiplicacion": #si el modo es multiplicacion, se multiplican intensidades.
        blended = (base * overlay) / 255.0
    elif fusion_mode == "adicion": #si el modo es adicion, se suman intensidades con recorte.
        blended = np.clip(base + overlay, 0, 255)
    elif fusion_mode == "diferencia": #si el modo es diferencia, se usa la diferencia absoluta.
        blended = np.abs(base - overlay)
    else: #si el modo es otro, se usa la segunda imagen como contenido base de fusion.
        blended = overlay

    result = base * (1.0 - alpha) + blended * alpha #mezcla la imagen base con el resultado del modo seleccionado
    return np.clip(result, 0, 255).astype(np.uint8) #limita el resultado al rango valido y se vuelve a uint8
```

Fusiona una segunda imágen a la imágen principal aplicando diferentes métodos de fusión siempre que estos se encuentren seleccionados, además de reescalando la segunda imágen al tamaño de la imágen principal

Reescalado de imágen

```python
def _resize_to_shape(self, image, width, height):
    """Redimensiona una imagen al ancho y alto requeridos."""
    resized = Image.fromarray(image).resize((width, height), Image.LANCZOS) #cambia la imágen de tamaño con Pillow
    return np.array(resized, dtype=np.uint8) #retorna la imágen redimensionada en formato de NumPy
```

Reescala la imágen dada al método haciendo uso de Pillow

Cálculo de luminancia

```python
def _calculate_luminance(self, image):
    """Calcula una luminancia aproximada a partir de RGB."""
    return ( #combina los canales RGB con sus pesos perceptuales
        0.299 * image[:, :, 0] #combina el canal rojo con su peso perceptual
        + 0.587 * image[:, :, 1] #combina el canal verde con mayor peso por sensibilidad visual.
        + 0.114 * image[:, :, 2] #combina el canal azul con el peso restante.
    )
```

Calcula la luminancia de una imágen a partir del peso perceptual de cada canal

Normalización de imágenes

```python
def _ensure_uint8_rgb(self, array):
    """Convierte cualquier imagen cargada a RGB uint8."""
    #si la imagen viene como float, se asume rango entre 0 y 1.
    if array.dtype == np.float32 or array.dtype == np.float64:
        array = np.clip(array, 0.0, 1.0) #limita el rango float para evitar valores invalidos
        array = (array * 255).astype(np.uint8) #escala al rango clasico de 0 a 255 y se convierte a uint8
    else: #si ya viene en enteros, solo se fuerza el rango permitido.
        array = np.clip(array, 0, 255).astype(np.uint8)

    #si la imagen es de un solo canal, se replica a tres canales RGB.
    if array.ndim == 2:
        array = np.stack([array, array, array], axis=-1) #se usa el mismo canal para rojo, verde y azul

    #si la imagen trae canal alfa, se elimina para trabajar solo con RGB.
    if array.ndim == 3 and array.shape[2] == 4:
        array = array[:, :, :3] #conserva solo los tres primeros canales (RGB), descartando alpha

    return array #retorna la imágen normalizada
```

Normaliza cualquier imágen otorgada al procesador de imágenes a formato uint8 para poder trabajar con las librerías utilizadas